Workbench es el mando. MySQL es el servidor.

Una academia nos acompañará en toda la guía: alumnos, cursos y matrículas. Nivel inicial · MySQL 8.0.16+ / 8.4, InnoDB · Workbench 8.0.

Workbench

Cliente con ventanas: envía órdenes y muestra respuestas.

mysql (CLI)

Cliente de terminal. Habla con el mismo servidor.



mysqld = proceso

Programa que está ejecutándose: acepta conexiones y gestiona datos.

Disco + memoria

InnoDB usa RAM, índices, archivos de datos y registros de recuperación.

EL SERVIDOR SIGUE TRABAJANDO

Cerrar la ventana ≠ apagar MySQL

Workbench puede cerrarse y mysqld seguir vivo como servicio del sistema. Un servidor puede alojar varias bases de datos; en MySQL, SCHEMA y DATABASE son equivalentes.

```
SELECT VERSION();
SHOW VARIABLES LIKE 'datadir';
SHOW DATABASES();
```

datadir es la carpeta del servidor. InnoDB guarda páginas binarias en tablespaces (habitualmente .ibd), además de logs. Un .sql guarda instrucciones; un .mwb, el modelo.

PRUÉBALO · Ejecuta las tres órdenes. ¿En qué máquina está la carpeta devuelta por datadir?

No hablamos HTTP con MySQL

SQL es el lenguaje de las órdenes. El protocolo MySQL define cómo viajan las peticiones y las respuestas.

Workbench

Interfaz gráfica

mysql

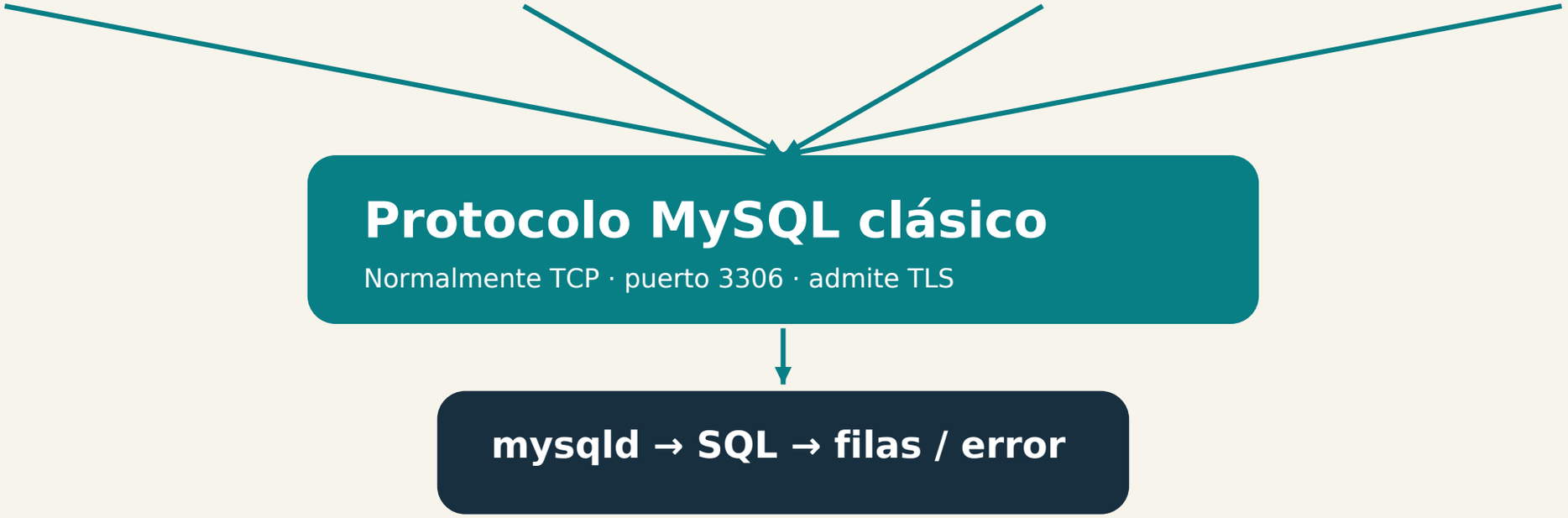
Consola / CLI

Django · Python

ORM + driver, p. ej. mysqlclient

Express · Node.js

Driver, p. ej. mysql2



¿Y la API web?

Navegador → HTTP(S) → Django/Express → driver → MySQL. Son dos conversaciones diferentes.

Conexión local

La CLI con localhost puede usar un socket Unix. 127.0.0.1 y --protocol=TCP fuerzan TCP en el ejemplo.

PRUÉBALO · El navegador pide /alumnos. Identifica quién interpreta HTTP y quién ejecuta SELECT.

Tu primera conexión, sin adivinar campos

Primero debe existir un servidor en marcha. Instalar Workbench por sí solo no instala ni arranca necesariamente MySQL Server.

Setup New Connection · esquema didáctico

Connection Name	Academia local (solo una etiqueta)
Connection Method	Standard (TCP/IP)
Hostname	127.0.0.1
Port	3306
Username	root (solo configuración inicial)
Default Schema	academia (opcional)

Test Connection → OK

Ruta por la GUI
Inicio → MySQL Connections → + → rellena los campos → Test Connection → introduce la contraseña → OK → abre la conexión.

```
mysql --protocol=TCP \
  -h 127.0.0.1 -P 3306 \
  -u root -p
```

Es una orden de terminal del sistema, no una sentencia SQL. -p pide la contraseña sin escribirla en la orden.

No conecta

Revisa servicio activo, host y puerto. Un fallo de red no se arregla cambiando SELECT.

Access denied

Revisa usuario, contraseña y host de la cuenta. Llegas al servidor, pero no entras.

Entrar no da permiso

Conectar y poder consultar una tabla son comprobaciones distintas.

PRUÉBALO · Abre una conexión y ejecuta SELECT CURRENT_USER(), DATABASE();

El editor SQL: escribe, ejecuta y lee

Usa el archivo de práctica para crear academia. Abre un script desde File → Open SQL Script y ejecútalo en tu conexión local.

Query 1 | Ejecutar selección/todo | Ejecutar sentencia actual | Stop

SCHEMAS

academia

  Tables

    alumno

    curso

    matricula

```
USE academia;
SELECT a.nombre, c.titulo, m.nota
FROM matricula AS m
JOIN alumno AS a ON a.id = m.alumno_id
JOIN curso AS c ON c.id = m.curso_id
ORDER BY a.nombre, c.titulo;
```

```
Result Grid: nombre       titulo       nota
             Ana          SQL          8.5

Action Output: éxito, error, filas y duración
```

1 · Qué ejecutas

Execute (All or Selection): selección o todo el editor.
Execute Current Statement: la sentencia bajo el cursor. Usa el menú Query o las ayudas de los botones.

2 · Qué miras

Result Grid contiene filas. Action Output muestra errores. Doble clic en academia la fija como esquema por defecto; Refresh actualiza el árbol.

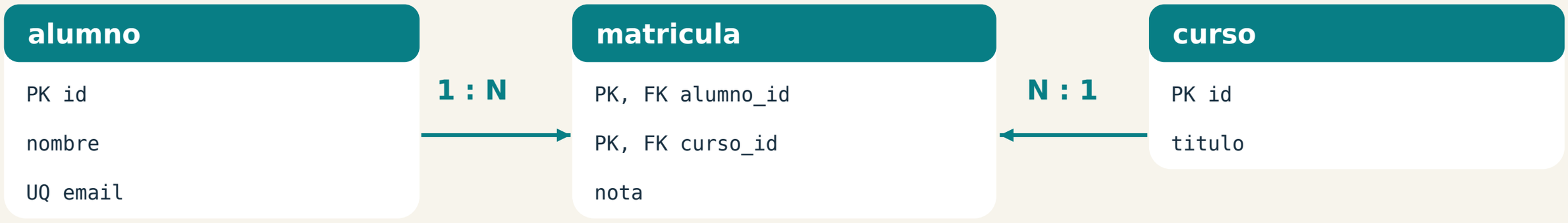
3 · Qué guardas

Guardar .sql guarda texto, no datos. El límite del editor puede recortar resultados. Editar una celda y pulsar Apply envía cambios al servidor.

PRUÉBALO · Selecciona solo el SELECT con sus JOIN. Ejecuta y compara Result Grid con Action Output.

De las tablas al modelo entidad-relación

Un diagrama EER de Workbench representa tablas, columnas y relaciones. Se obtiene de claves foráneas declaradas, no de nombres parecidos.



Un alumno puede tener 0..N matrículas; un curso, 0..N matrículas. Cada matrícula pertenece a un alumno y un curso. La pareja (alumno_id, curso_id) es la clave primaria: no se repite la inscripción.

Ya tengo tablas en el servidor

Database → Reverse Engineer
1. Elige la conexión y entra.
2. Selecciona academia.
3. Selecciona las tablas e importa.
4. Activa Place imported objects on a diagram, si aparece.
5. Abre el EER y guarda el modelo como .mwb.

Solo tengo el archivo SQL

File → Import → Reverse Engineer MySQL Create Script dentro de un modelo.

Si el lienzo está vacío, añade un diagrama y arrastra tablas desde Catalog. Unir columnas solo por su nombre no crea una FK. El modelo guardado no se actualiza en vivo.

PRUÉBALO · Genera el EER de academia. Localiza las dos FK y la clave primaria compuesta.

Del dibujo al SQL que sí crea las reglas

File → New Model → Add Diagram → coloca tablas. Doble clic en una tabla abre su editor. Un modelo es un diseño hasta que lo aplicas al servidor.

Las casillas esenciales

PK · clave primaria: identifica la fila.
NN · NOT NULL: valor obligatorio.
UQ · índice único: evita duplicados.
AI · AUTO_INCREMENT: genera IDs.

No marques UQ en nombre: dos alumnos pueden llamarse igual.

Trucos de modelado

En Foreign Keys, elige la tabla padre y mapea columna hija → columna padre.

Usa tipos compatibles, incluido UNSIGNED. Aquí cada FK forma parte de la PK: relación identificadora. En caso contrario, sería no identificadora.

Database → Forward Engineer: revisa SQL y destino antes de ejecutar.

```
CREATE DATABASE academia
  CHARACTER SET utf8mb4;
USE academia;
CREATE TABLE alumno (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(80) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE
) ENGINE=InnoDB;
CREATE TABLE curso (
  id INT PRIMARY KEY AUTO_INCREMENT,
  titulo VARCHAR(100) NOT NULL
) ENGINE=InnoDB;
CREATE TABLE matricula (
  alumno_id INT NOT NULL,
  curso_id INT NOT NULL,
  nota DECIMAL(3,1) NOT NULL,
  PRIMARY KEY (alumno_id, curso_id),
  CONSTRAINT fk_alumno FOREIGN KEY (alumno_id)
    REFERENCES alumno(id) ON DELETE RESTRICT,
  CONSTRAINT fk_curso FOREIGN KEY (curso_id)
    REFERENCES curso(id) ON DELETE RESTRICT,
  CONSTRAINT ck_nota CHECK (nota BETWEEN 0 AND 10)
) ENGINE=InnoDB;
```

PRUÉBALO · En matricula marca PK en las dos columnas ID y revisa el orden en la pestaña Indexes.

El guardián no deja pasar datos incoherentes

Las comprobaciones viven en el servidor y en su motor. Se aplican también si escribes desde CLI, Django o Express; no dependen de Workbench.



«¡Alto! Esa operación rompe una regla declarada.»

En InnoDB, las FK se comprueban durante la operación; no se aplazan hasta COMMIT.

PK · una matrícula por pareja

```
INSERT INTO matricula VALUES (1, 1, 9);
```

Si (1,1) ya existe → duplicado (1062).

FK · el alumno debe existir

```
INSERT INTO matricula VALUES (999, 1, 7);
```

Si no existe alumno 999 → referencia inválida (1452).

CHECK · la nota debe ser válida

```
UPDATE matricula SET nota=11 WHERE alumno_id=1;
```

Fuera de 0..10 → CHECK incumplido (3819).

NN + UQ · obligatorio y único

```
UPDATE alumno SET email=NULL WHERE id=1;
```

NULL → error (1048). Repetir otro email → 1062.

RESTRICT · no dejes huérfanos

```
DELETE FROM alumno WHERE id=1;
```

Si tiene matrículas → borrado bloqueado (1451).

Matices: CHECK se aplica desde MySQL 8.0.16. Un CHECK admite UNKNOWN (por NULL): por eso añadimos NOT NULL. UNIQUE permite varios NULL en columnas anulables. CASCADE borraría hijos automáticamente; úsalo solo si esa es la regla del negocio.

PRUÉBALO · Ejecuta cada error por separado en una BD de práctica. Explica la regla antes de leer el mensaje.

La misma puerta, distintas llaves

Una cuenta MySQL se identifica como usuario@host. El host indica desde dónde puede entrar; no es el nombre de la base de datos.

lector · solo consultar

Puede SELECT sobre academia. No puede INSERT, UPDATE, DELETE ni administrar usuarios.

app · trabajar con datos

Puede consultar y cambiar datos. No necesita privilegios para borrar tablas ni crear usuarios.

admin · administrar

Configura cuentas y privilegios. La cuenta de una aplicación no debería ser root.

```
-- Ejecuta como administrador
CREATE USER 'lector'@'localhost'
  IDENTIFIED BY 'Cambia-Esta-Clave-2026!';
GRANT SELECT ON academia.*
  TO 'lector'@'localhost';

CREATE USER 'app'@'localhost'
  IDENTIFIED BY 'Otra-Clave-Larga-2026!';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON academia.* TO 'app'@'localhost';
SHOW GRANTS FOR 'lector'@'localhost';
```

GUI equivalente
Server → Users and Privileges → Add Account.

Login: usuario, host y contraseña.
Schema Privileges: Add Entry → academia → marca solo lo necesario → Apply.

Crea otra conexión Workbench con lector y comprueba qué puede hacer.

¿Solo algunas columnas? Puedes conceder privilegios por columna o usar una vista con acceso limitado. Para restringir filas, diseña una vista o una política en la aplicación: GRANT SELECT sobre una tabla no filtra automáticamente «las filas de cada alumno».

PRUÉBALO · Entra como lector: SELECT funciona; un INSERT debe ser rechazado aunque sus datos sean válidos.

Cambiar la clave no es editar el llavero

La contraseña de la cuenta está en el servidor. Workbench puede guardar una credencial local para recordarla: son cosas distintas.

Desde Workbench · con privilegios

Server → Users and Privileges → elige la cuenta y su host → Login → Password y Confirm Password → Apply.

Después actualiza la contraseña guardada de la conexión (vault/keychain) y reconecta. Guardar una clave en Workbench no la cambia en MySQL.

```
-- Cambia tu propia contraseña
ALTER USER USER()
  IDENTIFIED BY 'Nueva-Clave-Larga-2026!';

-- Admin: cuenta concreta
ALTER USER 'lector'@'localhost'
  IDENTIFIED BY 'Otra-Clave-Larga-2026!';
```

Para cambiar datos con marcha atrás

```
START TRANSACTION;
UPDATE matricula SET nota=9
WHERE alumno_id=1 AND curso_id=1;
SELECT * FROM matricula;
ROLLBACK; -- prueba sin guardar
-- COMMIT; confirma en otra prueba
```

Autocommit suele estar activo: cada sentencia se confirma automáticamente fuera de una transacción explícita.

COMMIT confirma; ROLLBACK deshace cambios pendientes de datos en InnoDB. Si una sentencia falla, revisa la transacción; no supongas que todo se deshizo.

CREATE / ALTER / DROP suelen provocar commit implícito. No los uses para ensayar ROLLBACK.

PRUÉBALO · Antes de cambiar una clave, ejecuta SELECT USER(), CURRENT_USER(); e identifica la cuenta autenticada.

Inspeccionar, respaldar y recuperar

Workbench ofrece atajos visuales a tareas del servidor. Algunas requieren permisos especiales y funciones locales dependen del sistema operativo.

Panel de control

Server Status: versión, conexiones y actividad.
Client Connections: sesiones y consultas.
Performance Dashboard: carga y tráfico.
Startup / Shutdown: requiere gestión del servicio configurada; no basta con poder ejecutar SQL.

```
SHOW FULL PROCESSLIST;
SHOW CREATE TABLE academia.matricula;
SHOW INDEX FROM academia.matricula;
EXPLAIN SELECT * FROM academia.alumno
WHERE email='ana@example.com';
```

EXPLAIN muestra cómo se planea buscar. Un índice puede acelerar lecturas, pero consume espacio y trabajo al escribir.

Copia lógica desde la GUI

Server → Data Export → elige academia → estructura y datos → archivo .sql → Start Export.

Data Import/Restore restaura el SQL. Prueba siempre la restauración en otra BD o instancia de laboratorio. Exportar un CSV no equivale a una copia completa.

```
# Terminal del sistema
mysqldump -h 127.0.0.1 -u root -p \
  --single-transaction academia \
  > academia.sql
mysql -h 127.0.0.1 -u root -p \
  academia_restaurada < academia.sql
```

Antes de restaurar por CLI: crea academia_restaurada vacía. El volcado del ejemplo no usa --databases: sus tablas se cargarán en la BD indicada. --single-transaction permite una copia consistente de datos InnoDB; evita cambios DDL durante el volcado y revisa qué objetos adicionales necesitas incluir.

Diagnóstico útil: error 1046 → USE academia; 1146 → tabla/esquema incorrectos; 1175 → revisa el WHERE y la clave con Safe Updates. No desactives protecciones por rutina.

PRUÉBALO · Haz una copia, restáurala en academia_restaurada y compara el número de filas.

Local, nube o Docker: sigue siendo un servidor

Cambia dónde vive mysqld y cómo llegas a él. La consulta SQL puede ser la misma, pero la red, las credenciales y la persistencia deben estar bien configuradas.



En la nube · IONOS, AWS o Google Cloud

Workbench en tu portátil → red protegida → endpoint de la instancia → mysqld → disco remoto.

En Hostname pon el DNS real que te da el proveedor, por ejemplo db.aula.example; no escribas «aws» o «1and1». Configura puerto, usuario y TLS con verificación de certificado, o un túnel SSH. Autoriza tu IP / red; evita abrir 3306 a todo Internet.

TU ORDENADOR

Contenedor: mysql-aula

mysqld · puerto interno 3306

/var/lib/mysql → volumen

Workbench → 127.0.0.1:3307

```
# Terminal: Docker ya instalado
docker volume create mysql_aula_datos
docker run -d --name mysql-aula \
  -p 127.0.0.1:3307:3306 \
  -e MYSQL_ROOT_PASSWORD='Clave-Larga-2026!' \
  -v mysql_aula_datos:/var/lib/mysql \
  mysql:8.4
docker exec -it mysql-aula mysql -u root -p
```

```
CREATE USER 'aula'@'%' IDENTIFIED BY 'Clave-Aula-2026!';
GRANT ALL ON academia.* TO 'aula'@'%';
```

3307:3306 = puerto del anfitrión : puerto del contenedor. Con docker exec entra como root y crea aula con las dos órdenes SQL de arriba: es una cuenta de laboratorio con permisos sobre academia. En Workbench usa aula y puerto 3307. % admite hosts de la red Docker; el puerto publicado queda limitado al equipo local. Entre contenedores usa el nombre del servicio. El volumen persiste al borrar el contenedor. La inicialización solo actúa con el directorio vacío.

Sí hay escala horizontal. Tiene un precio.

«Relacional = solo vertical» es demasiado tajante. La dificultad depende de qué repartes: lecturas, disponibilidad, almacenamiento o escrituras.

VERTICAL

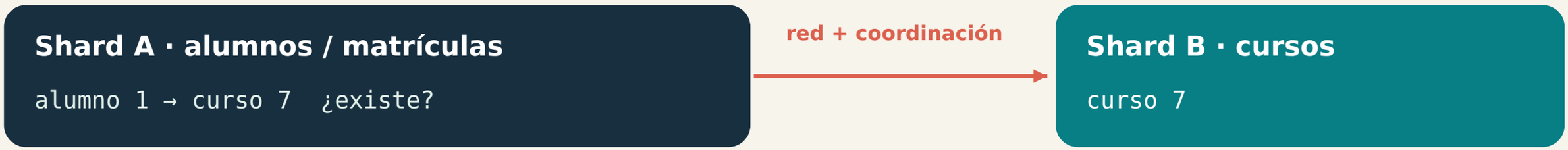
Una máquina más potente: más RAM, CPU o disco. Sencillo de entender; tiene límites físicos y de coste.

RÉPLICAS

Copias del mismo conjunto de datos: reparten lecturas y dan disponibilidad. Pueden ir con retraso; no reparten por sí solas datos ni escrituras.

SHARDING

Cada máquina guarda una parte. Reparte datos y escrituras; obliga a elegir una clave de partición y un enrutamiento.



El problema de las referencias: si padre e hijo viven en nodos distintos, comprobar la FK, unir tablas o confirmar una operación conjunta exige comunicación y coordinación. Hay latencia, fallos parciales, conflictos y posibles transacciones distribuidas. Colocar juntos los datos relacionados reduce ese coste.

Opciones reales: InnoDB Cluster / Group Replication aporta disponibilidad y réplicas; el modo multi-primary tiene restricciones y no es sharding. Vitess reparte MySQL en shards: sus FK deben quedar dentro del mismo shard. NDB Cluster es otra arquitectura, con compromisos propios.

Comparación justa: MongoDB tampoco evita el coste de las relaciones o las transacciones entre shards. Para aprender: buen modelo e índices → medir → escalar vertical / réplicas según la carga → plantear sharding cuando el problema lo justifique.